Weather Forecast Tracker API

COM661 Full Stack Strategies and Development, Assignment 1 (back-end).

A REST API built with **Python Flask** and **MongoDB**. Users browse weather forecasts for 38 cities, record the weather they actually observed, follow cities on a personal tracker, and set threshold alerts. Analytics endpoints use MongoDB aggregation pipelines, including a forecast-versus-observed accuracy measure.

Features

- Individual user accounts with bcrypt-hashed passwords and JWT login (2-hour tokens, logout via a token blacklist with a TTL index)
- Roles: normal users and admins. Admins manage cities and forecasts. Users add and manage their own observations and alerts
- Full CRUD on top-level documents (locations) and on sub-documents (forecasts, observations)
- Pagination, multi-field filtering, text search and geospatial "nearby" search
- Aggregation pipelines: statistics, forecast accuracy, rankings, condition breakdown
- Personal tracker dashboard joined with the user's alerts (`$lookup`)
- Input validation with type and range checks, informative error messages and correct HTTP status codes
- Automated Postman test collection

Requirements

- Python 3.10 or newer
- MongoDB Community Server 4.4 or newer (running on `mongodb://127.0.0.1:27017`)
- Postman (to run the test collection)

Setup

```bash
```

```
# 1. Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate            # Windows
# source venv/bin/activate       # Mac / Linux

# 2. Install the libraries
pip install -r requirements.txt

# 3. Create a .env file in the project folder
MONGO_URI=mongodb://127.0.0.1:27017
SECRET_KEY=any-long-random-text
```

Load the data

Option A: import the submitted JSON files (recommended)

This keeps the original `_id` values, which the Postman environment relies on.

bash

```
mongoimport --db weatherDB --collection locations --file locations.json --jsonArray
mongoimport --db weatherDB --collection users     --file users.json     --jsonArray
mongoimport --db weatherDB --collection alerts    --file alerts.json    --jsonArray
mongoimport --db weatherDB --collection blacklist --file blacklist.json --jsonArray
```

Option B: rebuild from the live source

bash

```
python scripts/import_data.py
```

This downloads forecast and archive data from the free Open-Meteo API for 38 cities. It creates new `_id` values, so update `city_id` in the Postman environment afterwards (copy any `_id` from `GET /locations`).

Run the API

bash

```
python app.py
```

The server runs on **http://127.0.0.1:5001/api/v1** (port 5001 is used because port 5000 is blocked on some Windows machines).

Create an admin user

1. Register a user with `POST /api/v1/auth/register`.

2. In MongoDB Compass, open `weatherDB` then `users`, and change that user's `role` from `"user"` to `"admin"`.
3. Log in again to receive a token that carries the admin role.

Run the tests

1. In Postman, import `tests/postman_collection.json` and `tests/postman_environment.json`.
2. Select the **Weather Tracker API** environment.
3. Make sure the server is running, then click **Run** on the collection.

The collection creates its own test users (alice, bob, carol, password `password123`) and uses random dates, so it can be run repeatedly.

Project structure

```
app.py                  application factory, blueprint registration, JSON error
handlers
config.py               settings read from .env
db.py                   MongoDB connection, collections and indexes
decorators.py           @jwt_required and @admin_required
utils.py                ObjectId checking, JSON serialisation, pagination helper
validators.py           input validation for forecasts, observations and locations
blueprints/
    auth.py             register, login, logout
    locations.py        city CRUD, search, filtering, nearby
    forecasts.py        forecast sub-document CRUD
    observations.py     observation sub-document CRUD with ownership checks
    analytics.py        aggregation endpoints
    tracker.py          follow cities, personal dashboard
    alerts.py           alert CRUD and triggered alerts
scripts/import_data.py  builds the dataset from Open-Meteo
tests/                  Postman collection, environment and test run
```

Database design

Collection	Contents
`locations`	One document per city: name, country, region, GeoJSON `geo` point, elevation, tags, embedded `forecasts[]` and `observations[]`
`users`	username, email, hashed password, role, `tracked_locations[]`, units
`alerts`	user, location_id, metric, operator (`gt` or `lt`), threshold, active
`blacklist`	logged-out tokens, removed automatically by a TTL index

Indexes: 2dsphere on `geo`, text on `name` and `region`, `country`, unique `username` and `email`, TTL on `blacklist.expires`.

Endpoint summary

Base URL: `http://127.0.0.1:5001/api/v1`. "Auth" means a Bearer token is required.

#	Method	Endpoint	Auth	Description	Status codes
1	POST	`/auth/register`	No	Create a user	201, 400, 409
2	POST	`/auth/login`	No	Log in, returns a token	200, 401
3	POST	`/auth/logout`	Yes	Blacklist the current token	200, 401
4	GET	`/locations`	No	List cities with pagination, filters and text search	200, 400
5	GET	`/locations/nearby`	No	Cities within `radius_km` of `lat` and `lng`	200, 400
6	GET	`/locations/<id>`	No	One city with all its data	200, 400, 404
7	POST	`/locations`	Admin	Add a city	201, 400, 401, 403
8	PUT	`/locations/<id>`	Admin	Update a city	200, 400, 401, 403, 404
9	DELETE	`/locations/<id>`	Admin	Delete a city	204, 400, 401, 403, 404
10	GET	`/locations/<id>/forecasts`	No	Forecasts, optional `from` and `to` dates	200, 400, 404

#	Method	Endpoint	Auth	Description	Status codes
11	GET	`/locations/<id>/forecasts/<fid>`	No	One forecast	200, 400, 404
12	POST	`/locations/<id>/forecasts`	Admin	Add a forecast (one per date)	201, 400, 401, 403, 404, 409
13	PUT	`/locations/<id>/forecasts/<fid>`	Admin	Update a forecast	200, 400, 401, 403, 404
14	DELETE	`/locations/<id>/forecasts/<fid>`	Admin	Delete a forecast	204, 400, 401, 403, 404
15	GET	`/locations/<id>/observations`	No	Observations, optional `user` filter	200, 400, 404
16	GET	`/locations/<id>/observations/<oid>`	No	One observation	200, 400, 404
17	POST	`/locations/<id>/observations`	Yes	Add an observation (one per user per date)	201, 400, 401, 404, 409

#	Method	Endpoint	Auth	Description	Status codes
18	PUT	`/locations/<id>/observations/<oid>`	Owner or admin	Update an observation	200, 400, 401, 403, 404
19	DELETE	`/locations/<id>/observations/<oid>`	Owner or admin	Delete an observation	204, 400, 401, 403, 404
20	GET	`/locations/<id>/stats`	No	Average, highest, lowest temperature and total rain for a date range	200, 400, 404
21	GET	`/locations/<id>/accuracy`	No	Forecast versus observed temperature error	200, 400, 404
22	GET	`/analytics/hottest`	No	Cities ranked by average maximum temperature	200, 400
23	GET	`/analytics/rainiest`	No	Cities ranked by total rainfall	200, 400
24	GET	`/analytics/windiest`	No	Cities ranked by average wind speed	200, 400
25	GET	`/analytics/condition-breakdown`	No	Share of days per weather condition	200
26	POST	`/users/me/tracked/<id>`	Yes	Follow a city	201, 400, 401, 404
27	DELETE	`/users/me/tracked/<id>`	Yes	Unfollow a city	204, 400,

#	Method	Endpoint	Auth	Description	Status codes
					401, 404
28	GET	`/users/me/tracker`	Yes	Dashboard of followed cities with next forecast and alerts	200, 401, 404
29	POST	`/alerts`	Yes	Create an alert	201, 400, 401, 404
30	GET	`/alerts`	Yes	List my alerts	200, 401
31	PUT	`/alerts/<id>`	Yes	Update one of my alerts	200, 400, 401, 404
32	DELETE	`/alerts/<id>`	Yes	Delete one of my alerts	204, 400, 401, 404
33	GET	`/alerts/triggered`	Yes	My active alerts matched by an upcoming forecast	200, 401

Query parameters for `GET /locations`

`pn` (page), `ps` (page size), `country`, `region`, `tag` (repeatable), `q` (text search), `min_temp`, `max_temp`, `condition`, `date`, `sort` (`name` or `country`), `order` (`asc` or `desc`).

Data source and limitations

- Forecasts come from the Open-Meteo forecast API. Imported observations come from the Open-Meteo historical archive, so the accuracy endpoint compares a forecast model with archive reanalysis data and user-submitted observations. It is not an independent real-world verification.
- Tokens are checked against a blacklist on every request; there is no refresh-token flow.
- Admin accounts are created by editing a user's role in MongoDB.

AI use

AI assistance (Claude) was used while building this project. Prompts and responses are included in the Design and Development document.